Start coding or generate with AI.

Aim: To demonstrate various data preprossing data techniques for a given dataset

✅ 0. Creation and loading of different datasent in python

```
import pandas as pd

# Dataset with replaced names
data = {
    'Name': ['Aarav', 'Ishita', 'Kabir', 'Meera', 'Vivaan', 'Diya', 'Rohan', 'Anaya'],
    'Age': [17, 17, 18, 17, 18, 17, 17, 18],
    'Gender': ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'F'],
    'Marks': [90, 76, None, 74, 65, None, 71, 80]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display the new DataFrame
print("Modified Dataset with New Names:")
print(df)
```

```
Modified Dataset with New Names:
     Name  Age Gender  Marks
0   Aarav   17      M   90.0
1  Ishita   17      F   76.0
2   Kabir   18      M    NaN
3   Meera   17      F   74.0
4  Vivaan   18      M   65.0
5    Diya   17      F    NaN
6   Rohan   17      M   71.0
7   Anaya   18      F   80.0
```

✅ 1. Reshaping the Data

```
import pandas as pd

data = {
    'Name': ['Arya', 'Aahan', 'Aman', 'Abhishek', 'Charu', 'Tusk', 'Iron'],
    'Age': [17, 17, 18, 17, 18, 17, 17],
    'Gender': ['M', 'F', 'M', 'M', 'M', 'F', 'F'],
    'Marks': [90, 76, None, 74, 65, None, 71]
}

df = pd.DataFrame(data)

df
```

	Name	Age	Gender	Marks
0	Arya	17	M	90.0
1	Aahan	17	F	76.0
2	Aman	18	M	NaN
3	Abhishek	17	M	74.0
4	Charu	18	M	65.0
5	Tusk	17	F	NaN
6	Iron	17	F	71.0

Next steps: Generate code with df | View recommended plots | New interactive sheet

✅ 2. Filtering the Data

```
import pandas as pd

data = {
    'Name': ['Arya', 'Mista', 'Igris', 'Beru'],
    'Age': [17, 18, 17, 18],
    'Gender': ['M', 'F', 'M', 'M'],
    'Marks': [90, 76, 74, 65]
}

df = pd.DataFrame(data)

# Filter specific columns
filtered = df.filter(['Name'])
```

```
print("Filtered Data:")
print(filtered)
```

```
Filtered Data:
    Name
0   Arya
1  Mista
2  Igris
3   Beru
```

✅ 3. Merging The Data

```
# import module
import pandas as pd

print('first table')

# Creating DataFrame for Student Details
details = pd.DataFrame({
    'ID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    'NAME': ['Jagdish', 'Praveen', 'Harsh',
             'Punit', 'Rahul', 'Nikhio',
             'Sahil', 'Ayush', 'Dimitri', 'Mohan'],
    'BRANCH': ['CSE', 'CSE', 'CSE', 'CSE',
               'CSE', 'CSE', 'CSE', 'CSE', 'CSE', 'CSE']
})

# Printing details
print(details)

print('--------------------------------------------------')

print("second table")

# Creating DataFrame for Fees_Status
fees_status = pd.DataFrame({
    'ID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    'PENDING': ['5000', '250', 'NIL', '9000', '15000', 'NIL',
                '4500', '1800', '250', 'NIL']
})

# Printing fees_status
print(fees_status)

print('--------------------------------------------------')

print('Merging the data base on ID ')
merged_df = pd.merge(details, fees_status, on='ID')
print(merged_df)
```

```
first table
    ID     NAME BRANCH
0  101  Jagdish    CSE
1  102  Praveen    CSE
2  103    Harsh    CSE
3  104    Punit    CSE
4  105    Rahul    CSE
5  106   Nikhio    CSE
6  107    Sahil    CSE
7  108    Ayush    CSE
8  109  Dimitri    CSE
9  110    Mohan    CSE
--------------------------------------------------
second table
    ID PENDING
0  101    5000
1  102     250
2  103     NIL
3  104    9000
4  105   15000
5  106     NIL
6  107    4500
7  108    1800
8  109     250
9  110     NIL
--------------------------------------------------
Merging the data base on ID
    ID     NAME BRANCH PENDING
0  101  Jagdish    CSE    5000
1  102  Praveen    CSE     250
2  103    Harsh    CSE     NIL
3  104    Punit    CSE    9000
4  105    Rahul    CSE   15000
5  106   Nikhio    CSE     NIL
6  107    Sahil    CSE    4500
7  108    Ayush    CSE    1800
8  109  Dimitri    CSE     250
9  110    Mohan    CSE     NIL
```

✅ 4. Handle missing values with mean

```
import numpy as np
import pandas as pd

# A dictionary with list values including some NaNs
data = {
    'G1': [10, 20, 30, 40],
    'G2': [25, np.nan, np.nan, 29],
    'G3': [15, 14, 17, 11],
    'G4': [21, 22, 23, 25]
}

# Create DataFrame
df = pd.DataFrame(data)

# Compute mean of column G2 (ignores NaN)
mean_value = df['G2'].mean()

# Fill NaN values in G2 with the calculated mean
df.loc[:, 'G2'] = df['G2'].fillna(value=mean_value)


# Display the updated DataFrame
print('Updated DataFrame:')
print(df)
```

```
Updated DataFrame:
   G1    G2  G3  G4
0  10  25.0  15  21
1  20  27.0  14  22
2  30  27.0  17  23
3  40  29.0  11  25
```

✅ 5. Normalization

```
# Import necessary libraries
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Create a sample DataFrame
data = {
    'Math': [72, 81, 90, 65, 75],
    'Science': [88, 72, 95, 70, 85],
    'English': [78, 83, 88, 64, 76]
}

df = pd.DataFrame(data)
print("Original Data:\n", df)

# Create a MinMaxScaler object
scaler = MinMaxScaler()

# Fit and transform the data
normalized_data = scaler.fit_transform(df)

# Create a new DataFrame with normalized values
normalized_df = pd.DataFrame(normalized_data, columns=df.columns)

print("\nNormalized Data (Min-Max):\n", normalized_df)
```

```
Original Data:
    Math  Science  English
0    72       88       78
1    81       72       83
2    90       95       88
3    65       70       64
4    75       85       76

Normalized Data (Min-Max):
    Math  Science   English
0  0.28     0.72  0.583333
1  0.64     0.08  0.791667
2  1.00     1.00  1.000000
3  0.00     0.00  0.000000
4  0.40     0.60  0.500000
```